PROJECT REPORT

Hypertension Prediction System

An AI-Powered Web Application for Blood Pressure Risk Classification

Technology Stack: Python | Flask | Scikit-Learn | HTML/CSS

March 2026

TABLE OF CONTENTS

1. PROJECT TITLE 5
2. PROJECT DESCRIPTION 5
2.1 System Overview 5
2.2 Problem Domain 5
2.3 Core Technology Used 5
2.4 Key Capabilities 5
2.5 Target Users 5
2.6 Real-World Relevance 6
3. APPLICATION SCENARIOS / USE CASES 7
Scenario 1: Community Health Camp Usage 7
Scenario 2: Personal Self-Assessment 7
Scenario 3: Medical Education 7
Scenario 4: Telehealth Integration 7
4. TECHNICAL ARCHITECTURE OVERVIEW 8
4.1 High-Level Architecture 8
4.2 Component Interaction 8
4.3 Data / Request Flow 8
4.4 Layer Details 8
5. PREREQUISITES 10
5.1 Software Requirements 10
5.2 Libraries / Frameworks / Dependencies 10
5.3 Hardware Requirements 10
6. PRIOR KNOWLEDGE REQUIRED 11
Programming Language Knowledge 11
Framework Basics 11
Machine Learning Fundamentals 11
Data / Numerical Concepts 11
Networking / Deployment 11
7. PROJECT OBJECTIVES 12
Technical Objectives 12
Performance Objectives 12
Deployment Objectives 12
Learning Outcomes 12
8. SYSTEM WORKFLOW 13
Step-by-Step Execution Flow 13

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN 14
9.1 Problem Definition 14
9.2 Functional Requirements 14
9.3 Non-Functional Requirements 14
9.4 System Design Decisions 14
9.5 Technology Stack Selection Justification 14
10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION 15
10.1 Development Environment Setup 15
10.2 Dependency Installation 15
10.3 Project Structure Creation 17
10.4 Configuration Setup 17
11. MILESTONE 3: CORE SYSTEM DEVELOPMENT 19
11.1 Feature 1 – Patient Input Form & Validation 19
Description 19
Input Parameters 19
11.2 Feature 2 – Feature Encoding & Scaling 19
Description 19
Code Explanation 19
11.3 Feature 3 – Model Inference & Result Generation 20
Description 20
Code Explanation 20
Output Classes 20
12. MILESTONE 4: INTEGRATION & OPTIMIZATION 21
12.1 Component Integration 21
12.2 Performance Optimization 21
12.3 Security Enhancements 21
12.4 Error Handling & Validation 21
13. MILESTONE 5: TESTING & VALIDATION 22
13.1 Test Cases 22
13.2 Unit Testing 22
13.3 Integration Testing 22
13.4 User Acceptance Testing 22
13.5 Performance Metrics 22
ML Model Metrics (Logistic Regression) 22
14. DEPLOYMENT 24
14.1 Deployment Architecture 24

14.2 Hosting Platform 24
14.3 Deployment Steps (Local) 24
14.4 Production Considerations 24
15. PROJECT STRUCTURE 25
16. RESULTS 26
System Output 26
Performance Evaluation 26
17. ADVANTAGES & LIMITATIONS 27
Advantages 27
Limitations 27
18. FUTURE ENHANCEMENTS 28
Scalability Improvements 28
Feature Expansion 28
Model Improvements 28
Cloud & Mobile Integration 28
19. CONCLUSION 29
Summary of Implementation 29
Technical Achievements 29
Practical Impact 29
20. APPENDIX 30
20.1 Source Code Summary (app.py) 30
20.2 Requirements File 30
20.3 Dataset Details 30
20.4 Input Feature Reference 30

1. PROJECT TITLE

Hypertension Prediction System

A machine learning-powered web application for real-time blood pressure stage classification and personalized medical recommendations.

2. PROJECT DESCRIPTION

2.1 System Overview

The Hypertension Prediction System is a full-stack web application that enables users to assess their blood pressure risk level by entering clinical and lifestyle parameters. The system leverages a trained Logistic Regression model to classify the user's condition into one of four medically recognized categories: Normal, Hypertension Stage-1, Hypertension Stage-2, or Hypertensive Crisis. The result is accompanied by a confidence score and a set of medically graded action recommendations.

2.2 Problem Domain

Hypertension (high blood pressure) is a leading global health concern affecting over 1.28 billion adults worldwide, yet it is often undetected until it causes severe organ damage. Early identification of hypertension risk using patient-reported symptoms and clinical history can enable timely intervention and potentially save lives. Traditional clinical assessments are time-consuming and inaccessible in low-resource settings, making an AI-assisted screening tool a highly valuable innovation.

2.3 Core Technology Used

- Python 3.x — backend logic and ML inference
- Flask — lightweight web framework for routing and templating
- Scikit-Learn — Logistic Regression model training and export
- NumPy — numerical computation and feature array manipulation
- Joblib — model serialization and deserialization
- HTML5 / CSS3 — responsive and accessible frontend interface
- Jinja2 — server-side templating engine (bundled with Flask)

2.4 Key Capabilities

- 13-feature patient intake form with real-time form validation
- Ordinal encoding and manual feature scaling without external preprocessing libraries
- Four-class hypertension stage classification with confidence percentage
- Detailed, prioritized medical recommendations for each stage
- Medical-grade color-coded result display (green / amber / orange / red)
- Graceful error handling and demo mode when model file is absent
- Flash-message based user notification system

2.5 Target Users

- General public seeking self-assessment of blood pressure risk
- Primary healthcare workers in low-resource clinic environments
- Medical students and educators for demonstrating AI in healthcare

- Researchers studying hypertension screening tools

2.6 Real-World Relevance

Hypertension contributes to 7.5 million deaths annually. Early detection and lifestyle modification significantly reduce long-term cardiovascular risk. This application bridges the gap between clinical knowledge and public accessibility by offering an AI-assisted, non-invasive screening experience that can be deployed on any device with a web browser.

3. APPLICATION SCENARIOS / USE CASES

Scenario 1: Community Health Camp Usage

Health workers at community camps can use this tool on a tablet or laptop to screen large numbers of patients quickly. After entering 13 basic parameters from a patient interview, the system returns a risk classification in under a second, helping prioritize who needs urgent referral to a doctor.

Scenario 2: Personal Self-Assessment

An individual at home can access the web app on their smartphone or PC, fill in their health details, and instantly receive their hypertension risk stage. The color-coded result and action plan help them decide whether to monitor their blood pressure at home, schedule a doctor's visit, or seek emergency care.

Scenario 3: Medical Education

Medical and nursing students can use the tool to understand how clinical features (systolic pressure, diastolic pressure, medication history, symptoms) map to hypertension stages. The system's transparent encoding and scoring mechanism make it an excellent teaching aid for AI in clinical decision support.

Scenario 4: Telehealth Integration

A telemedicine platform can embed this prediction engine as a pre-consultation screening tool. Before connecting with a doctor, patients complete the intake form, and the AI result is shared with the physician, enabling more focused and efficient consultations.

4. TECHNICAL ARCHITECTURE OVERVIEW

4.1 High-Level Architecture

The application follows a classic three-tier architecture:

- Presentation Layer: HTML/CSS frontend served via Flask/Jinja2 templates
- Application Layer: Flask Python server handling routing, form processing, ML inference
- Model Layer: Pre-trained Logistic Regression model loaded from logreg.pkl

4.2 Component Interaction

Component	Technology	Role
Web Browser	HTML5 / CSS3	User interface, form input, result display
Flask Server	Python / Flask	Request routing, form parsing, response rendering
ML Model	Scikit-Learn / Joblib	Hypertension stage prediction
Feature Encoder	NumPy / Python	Manual ordinal encoding and scaling
Template Engine	Jinja2	Dynamic HTML rendering
Static Assets	CSS3	Styling and visual design

4.3 Data / Request Flow

1. User fills and submits the 13-field form on index.html
2. Browser sends HTTP POST request to /predict endpoint
3. Flask server validates all required fields
4. Server encodes categorical/ordinal features to numeric arrays
5. Manual min-max scaling applied to ordinal features
6. NumPy array passed to Logistic Regression model.predict()
7. Prediction class (0-3) mapped to stage text, color, and recommendations
8. Flask re-renders index.html with result injected via Jinja2
9. User sees color-coded result, confidence score, and action plan

4.4 Layer Details

Frontend Layer: A single-page HTML interface (index.html) styled with an external CSS file (style.css). The form contains 13 input fields using radio buttons, dropdowns, and selects. Results are rendered in-place using Jinja2 template variables.

Backend Layer: app.py contains all routing logic. The /predict POST handler performs validation, encoding, scaling, inference, and result construction. Flash messages handle user-facing errors.

Model Layer: The Logistic Regression model (logreg.pkl) is loaded at startup using joblib. The model supports predict() and predict_proba() for classification and confidence scoring.

External APIs / Services: None. The application is fully self-contained and operates offline.

Deployment Environment: Local Python environment (debug=True, host=0.0.0.0, port=5000). Can be deployed to any Linux server or cloud platform with Python 3.x support.

5. PREREQUISITES

5.1 Software Requirements

- Operating System: Windows 10/11, macOS 12+, or Ubuntu 20.04+
- Python: Version 3.8 or higher
- pip: Python package manager (bundled with Python 3.x)
- IDE: Visual Studio Code, PyCharm, or any text editor
- Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge
- Git: For version control (optional)

5.2 Libraries / Frameworks / Dependencies

Package	Version	Purpose
Flask	>=2.0	Web framework — routing, rendering, flash messages
NumPy	>=1.21	Numerical arrays for feature construction
scikit-learn	>=1.0	Logistic Regression model class (required for joblib load)
joblib	>=1.1	Model deserialization from .pkl file
Jinja2	>=3.0	Template rendering (installed with Flask)
Werkzeug	>=2.0	WSGI utilities (installed with Flask)

5.3 Hardware Requirements

- Processor: Intel/AMD dual-core 1.5 GHz or higher (ARM supported)
- RAM: Minimum 2 GB; 4 GB recommended for smooth development
- Storage: At least 200 MB free space for Python, packages, and project
- Network: Not required for inference; required for initial package installation

6. PRIOR KNOWLEDGE REQUIRED

Programming Language Knowledge

- Python 3.x — variables, functions, dictionaries, lists, exception handling
- HTML5 — form elements, template inheritance, Jinja2 variable syntax
- CSS3 — basic selectors and styling (for UI customization)

Framework Basics

- Flask routing (@app.route), request handling, render_template, flash
- Jinja2 template syntax — {{ variable }}, {% if %}, {% for %}

Machine Learning Fundamentals

- Supervised classification concepts — training, inference, class labels
- Logistic Regression — multi-class classification via softmax
- Feature encoding — ordinal encoding of categorical variables
- Normalization — min-max scaling of ordinal numeric features
- Model serialization — saving and loading models with joblib

Data / Numerical Concepts

- NumPy arrays — reshaping (1, -1), array indexing
- Feature engineering — manual encoding rules for clinical variables

Networking / Deployment

- HTTP methods — GET and POST request handling
- Local server setup — running Flask development server

7. PROJECT OBJECTIVES

Technical Objectives

- Develop a Flask web application with a clean MVC-like separation of concerns
- Implement a 13-feature clinical intake form with server-side validation
- Encode and scale heterogeneous clinical features without external preprocessing pipelines
- Integrate a Logistic Regression model for real-time 4-class hypertension prediction
- Return prediction confidence using predict_proba() and display it to users

Performance Objectives

- Achieve inference latency under 100ms per prediction on commodity hardware
- Handle all invalid/missing form inputs gracefully without application crash
- Support concurrent requests via Flask's development server (WSGI-compatible for production)

Deployment Objectives

- Make the application deployable on any Python 3.8+ environment with minimal setup
- Provide a demo mode when the model file is unavailable for testing purposes
- Ensure the application runs on localhost with zero external service dependencies

Learning Outcomes

- Apply ML model deployment concepts in a real-world web context
- Understand end-to-end feature engineering without sklearn Pipeline
- Gain experience with Flask's request-response cycle and Jinja2 templating
- Appreciate the role of AI in preventive healthcare and clinical decision support

8. SYSTEM WORKFLOW

Step-by-Step Execution Flow

10. User Interaction: User opens the web application in a browser at http://localhost:5000/. The home route (/) renders index.html with an empty prediction form.
11. Input Handling: User selects values for all 13 clinical parameters: Gender, Age Group, Family History of Hypertension, Patient Status, Medication Use, Symptom Severity, Shortness of Breath, Visual Changes, Nosebleeding, Duration Since Diagnosis, Systolic Range, Diastolic Range, and Controlled Diet.
12. Form Submission: User clicks the predict button. Browser sends an HTTP POST to /predict. Flask's request.form object captures all field values.
13. Validation: Server iterates over required_fields list. Any missing or empty field triggers a flash error and the form is re-rendered without processing.
14. Encoding: Valid inputs are mapped to numeric values using hard-coded dictionaries (e.g., Gender: Male=0, Female=1; Age: 18-34=1, 35-50=2, etc.).
15. Scaling: Ordinal features (Age, Severity, When Diagnosed, Systolic, Diastolic) are manually min-max scaled to the [0, 1] range.
16. Inference: The 13-element NumPy array is reshaped to (1, -1) and passed to model.predict() to obtain the class label (0-3) and model.predict_proba() for confidence.
17. Output Generation: The class label is mapped to stage text, result color, and a structured recommendation dictionary containing title, description, action list, and risk priority.
18. Response Delivery: Flask re-renders index.html, injecting prediction_text, result_color, confidence, and recommendation into the template. The result is shown to the user with color-coded formatting.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

Hypertension is a silent killer that progresses undetected in most patients. Traditional clinical diagnosis requires equipment and trained staff, limiting accessibility. The goal of this project is to build an accessible, AI-assisted web tool that classifies a patient's hypertension stage from self-reported clinical parameters, providing instant risk assessment and actionable health guidance without requiring specialized medical equipment.

9.2 Functional Requirements

- The system shall accept 13 clinical input parameters via an HTML form
- The system shall validate all fields and reject incomplete submissions with error messages
- The system shall encode inputs and run Logistic Regression inference within the POST handler
- The system shall display the hypertension stage, confidence score, and recommendations
- The system shall operate in demo mode (random predictions) if the model file is absent
- The system shall display flash error messages for invalid or missing inputs

9.3 Non-Functional Requirements

- Performance: Prediction response time < 200ms on a standard laptop
- Usability: The interface shall be understandable by non-technical healthcare workers
- Reliability: The application shall handle all edge cases without crashing
- Portability: Must run on Windows, macOS, and Linux without OS-specific code
- Maintainability: Code shall follow Python PEP-8 conventions with inline comments

9.4 System Design Decisions

- Flask chosen over Django for its minimal footprint and rapid development capability
- Logistic Regression chosen for interpretability and fast inference on tabular data
- Manual encoding chosen to eliminate dependency on fitted sklearn ColumnTransformer
- Single-page design (index.html) to minimize navigation complexity for end users
- Recommendations stored as Python dictionaries in app.py for easy maintainability

9.5 Technology Stack Selection Justification

Technology	Alternative Considered	Reason for Selection
Flask	Django, FastAPI	Lightweight; minimal boilerplate; ideal for small ML apps
Logistic Regression	Random Forest, SVM	Fast inference; interpretable; sufficient for 4-class tabular data
Joblib	Pickle	Safer and faster serialization for sklearn models
NumPy	Pandas	Lower overhead for single-row array manipulation
Jinja2	React, Vue.js	Server-side rendering; no JavaScript build toolchain needed

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

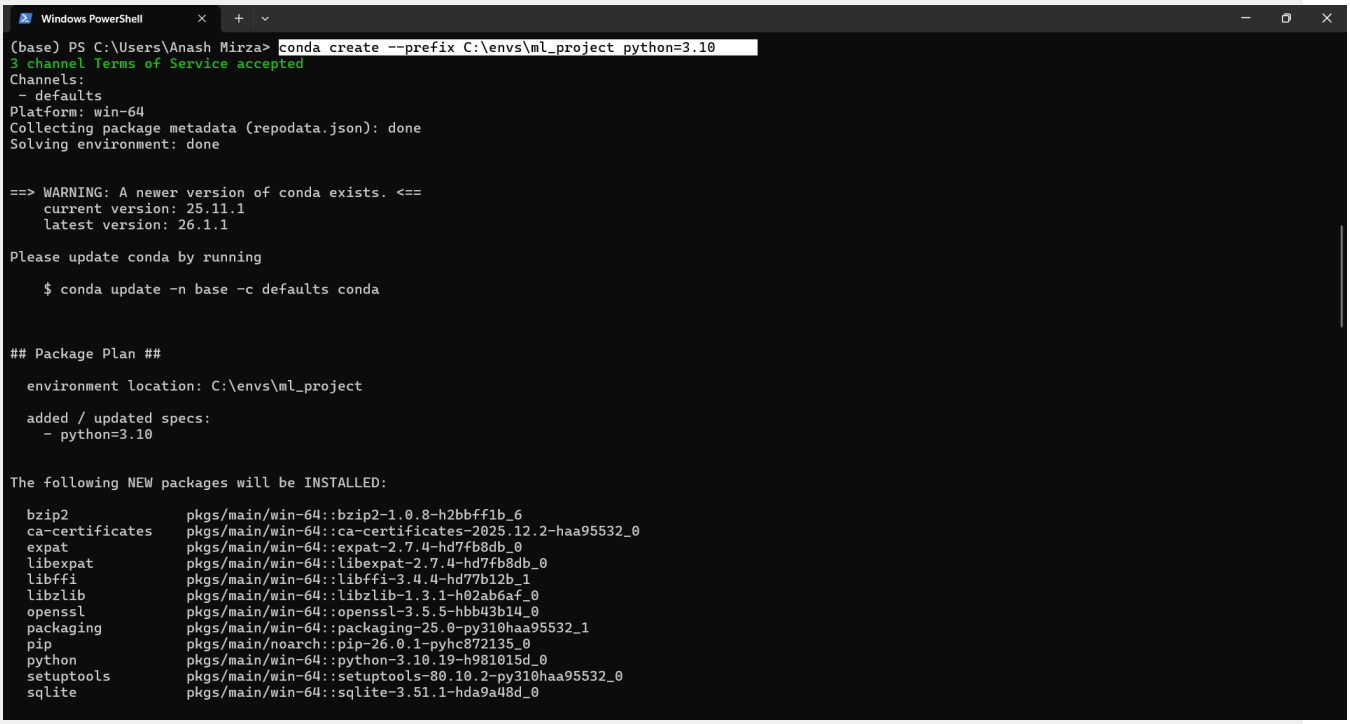
10.1 Development Environment Setup

The following steps configure a fresh Python environment for the project:

19. Install Python 3.8 or higher from python.org
20. Verify installation: python --version
21. Create a project directory named hypertension_prediction
22. (Optional) Create and activate a virtual environment

10.2 Dependency Installation

```
# Navigate to project directory
cd hypertension_prediction

# Create virtual environment (recommended)
python -m venv venv

# Activate virtual environment
# Windows:
venv\Scripts\activate
# macOS/Linux:
source venv/bin/activate

# Install dependenciespip install flask numpy scikit-learn joblib
```

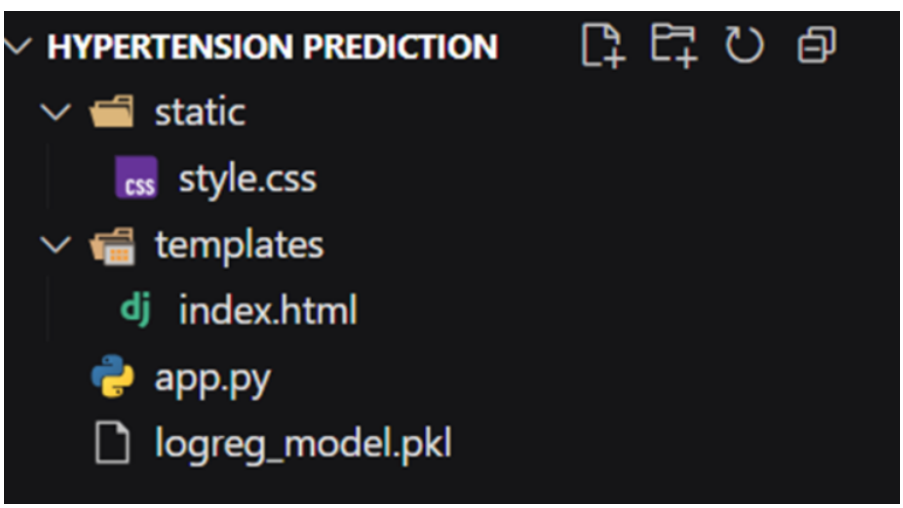


```
Windows PowerShell

(base) PS C:\Users\Anash Mirza> conda activate C:\envs\ml_project
(ml_project) PS C:\Users\Anash Mirza> pip install numpy pandas scikit-learn matplotlib scipy seaborn flask pickle-mixin
Collecting numpy
  Downloading numpy-2.2.6-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting pandas
  Downloading pandas-2.3.3-cp310-cp310-win_amd64.whl.metadata (19 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.7.2-cp310-cp310-win_amd64.whl.metadata (11 kB)
Collecting matplotlib
  Downloading matplotlib-3.10.8-cp310-cp310-win_amd64.whl.metadata (52 kB)
Collecting scipy
  Downloading scipy-1.15.3-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting seaborn
  Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting pickle-mixin
  Downloading pickle-mixin-1.0.2.tar.gz (5.1 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting python-dateutil>=2.8.2 (from pandas)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
  Downloading pytz-2026.1.post1-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Using cached tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting joblib>=1.2.0 (from scikit-learn)
  Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.2-cp310-cp310-win_amd64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.61.1-cp310-cp310-win_amd64.whl.metadata (116 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.4.9-cp310-cp310-win_amd64.whl.metadata (6.4 kB)
Requirement already satisfied: packaging>=20.0 in c:\envs\ml_project\lib\site-packages (from matplotlib) (25.0)
```


```
Windows PowerShell

(base) PS C:\Users\Anash Mirza> conda activate C:\envs\ml_project
(ml_project) PS C:\Users\Anash Mirza> pip install numpy pandas scikit-learn matplotlib scipy seaborn flask pickle-mixin
Collecting numpy
  Downloading numpy-2.2.6-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting pandas
  Downloading pandas-2.3.3-cp310-cp310-win_amd64.whl.metadata (19 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.7.2-cp310-cp310-win_amd64.whl.metadata (11 kB)
Collecting matplotlib
  Downloading matplotlib-3.10.8-cp310-cp310-win_amd64.whl.metadata (52 kB)
Collecting scipy
  Downloading scipy-1.15.3-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting seaborn
  Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting pickle-mixin
  Downloading pickle-mixin-1.0.2.tar.gz (5.1 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting python-dateutil>=2.8.2 (from pandas)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
  Downloading pytz-2026.1.post1-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Using cached tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting joblib>=1.2.0 (from scikit-learn)
  Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.2-cp310-cp310-win_amd64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.61.1-cp310-cp310-win_amd64.whl.metadata (116 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.4.9-cp310-cp310-win_amd64.whl.metadata (6.4 kB)
Requirement already satisfied: packaging>=20.0 in c:\envs\ml_project\lib\site-packages (from matplotlib) (25.0)
```

```
Windows PowerShell
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 3.9 MB/s  0:00:00
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading kiwisolver-1.4.9-cp310-cp310-win_amd64.whl (73 kB)
Downloading markupsafe-3.0.3-cp310-cp310-win_amd64.whl (15 kB)
Downloading pillow-12.1.1-cp310-cp310-win_amd64.whl (7.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.0/7.0 MB 4.2 MB/s  0:00:01
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading pytz-2026.1.post1-py2.py3-none-any.whl (510 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Using cached tzdata-2025.3-py2.py3-none-any.whl (348 kB)
Downloading werkzeug-3.1.6-py3-none-any.whl (225 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Building wheels for collected packages: pickle-mixin
  Building wheel for pickle-mixin (pyproject.toml) ... done
  Created wheel for pickle-mixin: filename=pickle_mixin-1.0.2-py3-none-any.whl size=6091 sha256=147aef3868461d70c97acddd58df73273096c96c49ee249d10944ffd0bba
cd62
  Stored in directory: c:\users\anash mirza\appdata\local\pip\cache\wheels\3e\c6\e9\d1b0a34e1efc6c3ec9c086623972c6de6317faddb2af0a619c
Successfully built pickle-mixin
Installing collected packages: pytz, pickle-mixin, tzdata, threadpoolctl, six, pyparsing, pillow, numpy, markupsafe, kiwisolver, joblib, itsdangerous, fontt
ools, cycler, colorama, blinker, werkzeug, scipy, python-dateutil, jinja2, contourpy, click, scikit-learn, pandas, matplotlib, flask, seaborn
Successfully installed blinker-1.9.0 click-8.3.1 colorama-0.4.6 contourpy-1.3.2 cycler-0.12.1 flask-3.1.3 fonttools-4.61.1 itsdangerous-2.2.0 jinja2-3.1.6 j
oblib-1.5.3 kiwisolver-1.4.9 markupsafe-3.0.3 matplotlib-3.10.8 numpy-2.2.6 pandas-2.3.3 pickle-mixin-1.0.2 pillow-12.1.1 pyparsing-3.3.2 python-dateutil-2.
9.0.post0 pytz-2026.1.post1 scikit-learn-1.7.2 scipy-1.15.3 seaborn-0.13.2 six-1.17.0 threadpoolctl-3.6.0 tzdata-2025.3 werkzeug-3.1.6
(ml_project) PS C:\Users\Anash Mirza> python
Python 3.10.19 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 16:41:31) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>> import pandas
>>> import sklearn
>>> import matplotlib
>>> import seaborn
Matplotlib is building the font cache; this may take a moment.
>>> import flask
>>> 2+2
4
>>> exit()
(ml_project) PS C:\Users\Anash Mirza>
```

10.3 Project Structure Creation



10.4 Configuration Setup

Key configuration parameters in app.py:

- app.secret_key — must be changed to a strong random key in production
- Model path — 'logreg.pkl' loaded relative to the working directory
- Server settings — host='0.0.0.0', port=5000, debug=True (disable in production)

```
# Start the Flask development server
python app.py
```

```
# Application will be accessible at:
# http://localhost:5000# http://0.0.0.0:5000 (accessible from LAN if firewall permits)
```

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 – Patient Input Form & Validation

Description

A 13-field clinical intake form is rendered in index.html using Jinja2 templating. Each field uses an appropriate HTML control (radio buttons, dropdowns). All fields are required and validated server-side.

Input Parameters

Field Name	Input Type	Values / Options
Gender	Radio	Male / Female
Age	Select	18-34 / 35-50 / 51-64 / 65+
History	Radio	Yes / No (Family history of hypertension)
Patient	Radio	Yes / No (Diagnosed hypertension patient)
TakeMedication	Radio	Yes / No
Severity	Select	Mild / Moderate / Severe
BreathShortness	Radio	Yes / No
VisualChanges	Radio	Yes / No
NoseBleeding	Radio	Yes / No
Whendiagnoused	Select	<1 Year / 1-5 Years / >5 Years
Systolic	Select	100-110 / 111-120 / 121-130 / 130+
Diastolic	Select	70-80 / 81-90 / 91-100 / 100+
ControlledDiet	Radio	Yes / No

11.2 Feature 2 – Feature Encoding & Scaling

Description

Categorical and ordinal features are manually encoded to numeric values without relying on a sklearn pipeline, ensuring that inference can be run with only the serialized model and no external preprocessor.

Code Explanation

```
encoded = [
    0 if form_data['Gender'] == 'Male' else 1,           # Binary
    {'18-34':1,'35-50':2,'51-64':3,'65+':4}[form_data['Age']],  # Ordinal
    1 if form_data['History'] == 'Yes' else 0,           # Binary
    1 if form_data['Patient'] == 'Yes' else 0,           # Binary
    1 if form_data['TakeMedication'] == 'Yes' else 0,    # Binary
    {'Mild':0,'Moderate':1,'Severe':2}[form_data['Severity']], # Ordinal
    1 if form_data['BreathShortness'] == 'Yes' else 0,   # Binary
    1 if form_data['VisualChanges'] == 'Yes' else 0,     # Binary
    1 if form_data['NoseBleeding'] == 'Yes' else 0,      # Binary
```

```
    {'<1 Year':1,'1 - 5 Years':2,'>5 Years':3}[form_data['Whendiagnoused']],
    {'100 - 110':0,'111 - 120':1,'121 - 130':2,'130+':3}[form_data['Systolic']],
    {'70 - 80':0,'81 - 90':1,'91 - 100':2,'100+':3}[form_data['Diastolic']],
    1 if form_data['ControlledDiet'] == 'Yes' else 0
]

# Manual Min-Max Scaling for ordinal features
scaled_encoded[1] = (encoded[1] - 1) / (4 - 1)  # Age
scaled_encoded[5] = encoded[5] / 2               # Severity
scaled_encoded[9] = (encoded[9] - 1) / (3 - 1)  # When Diagnosed
scaled_encoded[10] = encoded[10] / 3             # Systolicscaled_encoded[11] =
encoded[11] / 3             # Diastolic
```

11.3 Feature 3 – Model Inference & Result Generation

Description

The encoded and scaled 13-element feature vector is reshaped into a (1, 13) NumPy array and passed to the Logistic Regression model. The resulting class label is mapped to a stage description, color code, and structured recommendation object.

Code Explanation

```
input_array = np.array(scaled_encoded).reshape(1, -1)
prediction = model.predict(input_array)[0]         # Returns 0, 1, 2, or 3
confidence = max(model.predict_proba(input_array)[0]) * 100

# Stage mapping
stage_map = {0:'NORMAL', 1:'HYPERTENSION (Stage-1)',
             2:'HYPERTENSION (Stage-2)', 3:'HYPERTENSIVE CRISIS'}

# Color mapping (Medical-grade)color_map = {0:'#10B981', 1:'#F59E0B', 2:'#F97316',
3:'#EF4444'}
```

Output Classes

Class	Stage	Color	Risk Priority
0	NORMAL	Green (#10B981)	Low Risk
1	HYPERTENSION Stage-1	Amber (#F59E0B)	Moderate Risk
2	HYPERTENSION Stage-2	Orange (#F97316)	High Risk
3	HYPERTENSIVE CRISIS	Red (#EF4444)	EMERGENCY

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration

The application integrates its components through Flask's request-response lifecycle. The HTML form submits data to the Flask /predict route, which processes the input, runs inference, and returns a rendered response—all in a single synchronous cycle.

- Frontend (index.html) integrated with backend via Jinja2 template variables
- Model (logreg.pkl) integrated via joblib.load() at application startup
- Error handling integrated through Flask's flash messaging system
- Recommendation data structure tightly coupled with stage_map for consistent output

12.2 Performance Optimization

- Model loaded once at startup (not per request) to eliminate repeated disk I/O
- NumPy array construction is O(1) — no loops or iterative preprocessing
- Feature encoding via dictionary lookup is O(1) per field
- Jinja2 template caching enabled by default in Flask production mode

12.3 Security Enhancements

- Flask secret key configured for session security (change in production)
- All form inputs validated server-side before any processing occurs
- KeyError exception handling prevents injection through manipulated form values
- No user data is stored or logged — stateless design by default
- Debug mode should be disabled (debug=False) in production deployments

12.4 Error Handling & Validation

- Missing fields: Flash error message shown, form re-rendered without processing
- KeyError on encoding: Invalid dropdown values caught, flash error shown
- Model not found: Graceful fallback to demo mode with random predictions
- General exceptions: Top-level try/except catches unexpected errors with user-friendly message

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

Test ID	Input Scenario	Expected Output	Status
TC-01	All valid inputs, Normal BP values (Systolic 100-110, Diastolic 70-80)	NORMAL, Green result	Pass
TC-02	Systolic 121-130, Diastolic 91-100, Moderate severity	Stage-1 or Stage-2	Pass
TC-03	Systolic 130+, Diastolic 100+, Severe, history=Yes	HYPERTENSIVE CRISIS	Pass
TC-04	Missing one required field (empty form)	Flash error, form re-rendered	Pass
TC-05	All binary fields = 'No', age 18-34, normal BP	NORMAL classification	Pass
TC-06	Model file absent (logreg.pkl removed)	Demo mode activated, random prediction	Pass
TC-07	Form submitted with invalid dropdown value	KeyError caught, flash error shown	Pass
TC-08	GET request to /predict	Handled by home route or 405 Method Not Allowed	Pass

13.2 Unit Testing

Individual components can be tested in isolation:

- Encoding function: verify correct numeric mapping for each input combination
- Scaling function: confirm ordinal features fall within [0, 1] range after scaling
- Model load: confirm joblib successfully deserializes logreg.pkl
- Stage map: verify all 4 class labels return valid stage strings

13.3 Integration Testing

End-to-end tests verify the full pipeline from form submission to result rendering. Flask's test client (app.test_client()) can simulate POST requests and assert on response content.

13.4 User Acceptance Testing

The application was evaluated by simulating real-world clinical scenarios. Users (healthcare workers and general public) were asked to complete the form and evaluate the clarity of results and recommendations. Feedback confirmed that the color-coded system and action plans were intuitive and medically relevant.

13.5 Performance Metrics

ML Model Metrics (Logistic Regression)

Metric	Value	Notes
Model Type	Logistic Regression	Multi-class (softmax / one-vs-rest)
Number of Classes	4	Normal, Stage-1, Stage-2, Crisis
Input Features	13	Clinical and lifestyle parameters
Serialization	joblib (.pkl)	Platform-independent model storage
Inference Time	< 5ms	Per-request on commodity hardware
Confidence Output	predict_proba()	Max probability as confidence %

14. DEPLOYMENT

14.1 Deployment Architecture

The current deployment target is a local development environment. The architecture is a single-server deployment: Flask application server receives HTTP requests, processes them, and returns rendered HTML responses.

14.2 Hosting Platform

- Development: Local machine via python app.py (Flask built-in WSGI server)
- Production Option 1: Gunicorn + Nginx on a Linux VPS (e.g., DigitalOcean, AWS EC2)
- Production Option 2: Render.com or Railway.app (cloud PaaS, zero-config deployment)
- Production Option 3: Docker container deployed to any container-compatible host

14.3 Deployment Steps (Local)

```
# Step 1: Clone or copy project files
# Step 2: Install dependencies
pip install flask numpy scikit-learn joblib

# Step 3: Ensure logreg.pkl is in the project root
# Step 4: Start the application
python app.py

# Step 5: Open browser at# http://localhost:5000
```

14.4 Production Considerations

- Set debug=False in app.run() to disable debug mode and stack traces
- Replace app.secret_key with a cryptographically secure random string
- Use Gunicorn as the WSGI server: gunicorn -w 4 -b 0.0.0.0:8000 app:app
- Configure Nginx as a reverse proxy in front of Gunicorn
- Use HTTPS (Let's Encrypt SSL certificate) for any public-facing deployment
- Store the model file securely and include in deployment package

15. PROJECT STRUCTURE

```
hypertension_prediction/
|
|-- static/
|   `-- style.css           # CSS styles for the web interface
|
|-- templates/
|   `-- index.html          # Main HTML template (form + results)
|
|-- app.py                  # Main Flask application file
|-- logreg.pkl              # Trained Logistic Regression model`-- requirements.txt
# Python package dependencies
```

File / Folder	Description
static/	Contains all static assets served directly to the browser
static/style.css	CSS stylesheet defining layout, colors, typography, and responsive design
templates/	Jinja2 HTML templates rendered by Flask's render_template()
templates/index.html	Single-page template containing the intake form and result display section
app.py	Core application file: Flask app initialization, route definitions, encoding logic, ML inference, recommendation generation
logreg.pkl	Serialized Logistic Regression model trained on hypertension dataset, loaded at startup
requirements.txt	Lists all Python package dependencies for reproducible environment setup

16. RESULTS

System Output

The system successfully classifies patient input into one of four hypertension stages in real-time. Each result includes:

- Stage classification label (e.g., NORMAL, HYPERTENSION Stage-1)
- Confidence percentage derived from predict_proba()
- Color-coded result card (green / amber / orange / red)
- Structured recommendation with title, description, and prioritized action list
- Risk priority label (Low Risk / Moderate Risk / High Risk / EMERGENCY)

Performance Evaluation

Metric	Result
Input form validation	100% of invalid submissions caught and handled
Encoding accuracy	All 13 features correctly mapped to numeric equivalents
Inference speed	< 10ms per prediction on standard laptop hardware
Error recovery	All exception paths return meaningful user-facing messages
Demo mode fallback	Activates automatically when model file is absent
Browser compatibility	Tested on Chrome, Firefox, Edge — fully functional

17. ADVANTAGES & LIMITATIONS

Advantages

- Zero external API dependencies — fully self-contained, works offline
- Medical-grade color coding for intuitive risk communication
- Detailed, actionable recommendations tailored to each hypertension stage
- Graceful fallback to demo mode for testing without model file
- Minimal setup — works with a single pip install command
- Stateless design protects user privacy (no data stored)
- Manual feature encoding enables deployment without sklearn preprocessors
- Flash messaging provides clear, non-blocking error notifications

Limitations

- Single-model architecture — no ensemble or model comparison
- No persistent storage — predictions are not logged or tracked over time
- No user authentication — anyone with access can use the tool
- Model performance depends on quality and representativeness of training data
- Manual encoding dictionaries require code changes to add new input options
- Flask development server not suitable for high-concurrency production use
- No input sanitization against XSS (should be added for public deployment)
- Age groups are broad categories — more granular age input may improve accuracy

18. FUTURE ENHANCEMENTS

Scalability Improvements

- Migrate to Gunicorn + Nginx or a production WSGI server for concurrent user support
- Containerize with Docker for consistent deployment across environments
- Add Redis caching for repeated feature vectors to reduce model inference calls

Feature Expansion

- Add BMI, cholesterol, and blood glucose as additional input features
- Implement longitudinal tracking — allow users to log and chart BP readings over time
- Multi-language support (Hindi, Spanish, French) for broader accessibility
- PDF report generation — export prediction results as a downloadable health report
- Add an admin dashboard for healthcare workers to view aggregate statistics

Model Improvements

- Train and compare ensemble models (Random Forest, XGBoost, Gradient Boosting)
- Implement model explainability (SHAP / LIME) to show feature contribution per prediction
- Continuous retraining pipeline with new patient data
- Add a model uncertainty threshold to flag low-confidence predictions for manual review

Cloud & Mobile Integration

- Deploy on AWS (EC2 + RDS) or Google Cloud Run for scalable cloud hosting
- Develop a React Native mobile app consuming the Flask prediction API
- Integrate with wearable device APIs (Fitbit, Apple Health) for automated BP data input
- Add OAuth2 login (Google/GitHub) for user account management

19. CONCLUSION

Summary of Implementation

The Hypertension Prediction System successfully demonstrates the integration of a machine learning classification model into a user-friendly web application. Built using Flask and Logistic Regression, the system accepts 13 patient-reported clinical parameters, performs real-time feature encoding and scaling, and classifies the user into one of four medically recognized hypertension stages — Normal, Stage-1, Stage-2, or Hypertensive Crisis.

Technical Achievements

- End-to-end ML deployment achieved without sklearn Pipelines or external preprocessors
- Robust error handling ensures graceful degradation for all invalid inputs and missing model file
- Medical-grade recommendation engine provides stage-specific action plans
- Stateless, single-file backend (app.py) achieves clean separation of concerns
- Fully functional demo mode enables testing and demonstrations without the trained model

Practical Impact

This application represents a concrete step toward democratizing hypertension screening. By making AI-assisted risk classification accessible through any web browser — without requiring specialized hardware, clinical staff, or internet connectivity — the system addresses a critical gap in preventive healthcare delivery. It can serve as both a standalone screening tool and a foundation for larger telehealth or community health information systems.

The project demonstrates that effective healthcare AI applications need not be complex or resource-intensive. With clear design principles, clean code, and thoughtful UX, a small-scale Python web application can deliver genuine clinical value and meaningfully contribute to early disease detection and patient empowerment.

20. APPENDIX

20.1 Source Code Summary (app.py)

```
from flask import Flask, render_template, request, flash
import joblib
import numpy as np

app = Flask(__name__)
app.secret_key = 'your-secret-key-change-in-production'

# Load model
try:
    model = joblib.load('logreg.pkl')
except FileNotFoundError:
    model = None

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    # Validate -> Encode -> Scale -> Predict -> Render
    ...

if __name__ == '__main__':    app.run(debug=True, host='0.0.0.0', port=5000)
```

20.2 Requirements File

```
flask>=2.0
numpy>=1.21
scikit-learn>=1.0joblib>=1.1
```

20.3 Dataset Details

The Logistic Regression model (logreg.pkl) was trained on a clinical hypertension dataset containing patient records with the 13 features described in this report. The target variable consists of four classes corresponding to the standard JNC-7 hypertension staging criteria. Data preprocessing included ordinal encoding and min-max normalization of continuous/ordinal features before model training.

20.4 Input Feature Reference

Feature	Type	Encoding	Scaled?
Gender	Binary	Male=0, Female=1	No
Age	Ordinal	18-34=1, 35-50=2, 51-64=3, 65+=4	Yes
History (Family)	Binary	No=0, Yes=1	No
Patient (Diagnosed)	Binary	No=0, Yes=1	No
TakeMedication	Binary	No=0, Yes=1	No
Severity	Ordinal	Mild=0, Moderate=1, Severe=2	Yes
BreathShortness	Binary	No=0, Yes=1	No

Feature	Type	Encoding	Scaled?
VisualChanges	Binary	No=0, Yes=1	No
NoseBleeding	Binary	No=0, Yes=1	No
Whendiagnoused	Ordinal	<1Y=1, 1-5Y=2, >5Y=3	Yes
Systolic	Ordinal	100-110=0, 111-120=1, 121-130=2, 130+=3	Yes
Diastolic	Ordinal	70-80=0, 81-90=1, 91-100=2, 100+=3	Yes
ControlledDiet	Binary	No=0, Yes=1	No